Introduction to Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation, or RAG, is a technique that lets a large language model answer questions using information from external documents. Instead of relying only on what the model learned during training, a RAG system first searches a knowledge base for relevant passages and then gives those passages to the model as context.

RAG is useful because language models can be out of date, can invent facts, and do not know about private documents such as company manuals, research papers, or personal notes. By grounding answers in retrieved text, RAG reduces hallucinations and makes answers easier to verify.

A typical RAG application is a chatbot that answers questions about a PDF, a help center, or a set of internal reports.

How a RAG Pipeline Works

A RAG pipeline has five main steps. Step 1 is loading: documents such as PDF files are read and converted into plain text. Step 2 is splitting: the text is divided into small overlapping chunks, usually between 500 and 1000 characters, so each chunk holds one focused idea.

Step 3 is embedding: each chunk is converted into a list of numbers called a vector that captures its meaning. Step 4 is storage: the vectors are saved in a vector database such as FAISS or Chroma. Step 5 is retrieval and generation: when a user asks a question, the question is embedded, the most similar chunks are fetched, and the language model writes an answer using only those chunks.

The two most important settings are chunk size and the number of chunks retrieved, often called k. A common starting point is a chunk size of 1000, an overlap of 150, and k equal to 4.

Tools Commonly Used for RAG

LangChain is a Python framework that connects document loaders, text splitters, embedding models, vector stores, and language models into one pipeline. It is popular for building RAG applications quickly.

FAISS is a library from Meta for fast similarity search over vectors and runs locally on a normal computer. Chroma is another local vector database that also saves data to disk.

For the language model, developers can use hosted models such as Gemini or GPT, or free local models run through Ollama, such as Llama 3.2. For embeddings, options include Gemini embeddings, OpenAI embeddings, and the local nomic-embed-text model.

Common Problems and Tips

If answers are wrong or vague, the cause is usually poor retrieval. Try smaller chunks, a larger k, or a better embedding model. If the model ignores the context, make the prompt stricter by telling it to answer only from the provided context and to say it does not know otherwise.

Scanned PDFs contain images instead of text, so they need OCR before they can be used. Very large PDFs can be slow to load and may hit rate limits on free API tiers, so test with a small file first.

To check that a RAG system works, ask questions whose answers you already know from the document, such as: What are the five steps of a RAG pipeline? or What chunk size is a common starting point?